

CAPSTONE PROJECT REPORT

AUTOMATION TESTING FRAMEWORK FOR TEST AUTOMATION PRACTICE WEBSITE

Submitted By

P KARUNAKAR



Guided By :

Ms. VAISHALI SONAWANE

SDET Mentor



WIPRO TALENTNEXT PROGRAM

2026

INDEX

Sl.No	Topic	Page No
1	Project Title	1
2	Project Overview	1
3	Project Objectives	1
4	Technologies Used	2
5	Framework Design	3
6	Features Implemented	3 - 5
7	Automated Test Cases	5 - 7
8	Automation Test Report	7
9	Defects Identified	8
10	Defect Report	8 - 12
11	Defect Summary Table	12
12	Jenkins Integration	13 - 14
13	GitHub Repository	15
14	Test Execution Summary	15
15	Challenges Faced	16
16	Conclusion	16

Project Title:

Automation Testing Framework for Test Automation Practice Website Using Selenium, TestNG, Maven, Jenkins, and GitHub

Project Overview:

This project focuses on automating the Test Automation Practice Website using Selenium WebDriver with Java. The framework is designed using the Page Object Model (POM) design pattern and integrates TestNG, Maven, Jenkins, Apache POI, and GitHub for efficient test automation, reporting, and continuous integration.

The framework automates major web functionalities including form validation, radio buttons, checkboxes, dropdowns, alerts, file uploads, window handling, tables, broken link verification, mouse actions, and date picker validation.

Project Objectives:

- Automate repetitive manual test cases.
- Improve test coverage and execution speed.
- Implement a reusable Page Object Model framework.
- Integrate TestNG for test management.
- Read test data using Apache POI.
- Generate execution reports.
- Integrate GitHub for version control.
- Integrate Jenkins for Continuous Integration.
- Reduce manual testing effort and improve quality.

Technologies Used:

Tool	Purpose
Java	Programming Language
Selenium WebDriver	Web Automation
TestNG	Test Execution Framework
Maven	Build Management
Apache POI	Excel Data Handling
Jenkins	Continuous Integration
GitHub	Source Code Management
Extent Reports	Reporting
Chrome Browser	Test Execution

Project Website:

<https://testautomationpractice.blogspot.com/>

Framework Design:

The framework follows the Page Object Model (POM) design pattern.

Project Structure:

CapstoneProject

```
├── src/test/java
|   ├── base
|   ├── pages
|   ├── testcases
|   ├── listeners
|   └── utilities
└── src/test/resources
    ├── pom.xml
    ├── testng.xml
    └── Jenkinsfile
```

Features Implemented :

Selenium WebDriver

- Browser Launch
- Element Identification
- Form Handling
- Validation

TestNG

- Test Prioritization
- Assertions
- Test Execution
- Reporting

Apache POI

- Excel Data Reading
- Data Driven Testing

Alerts Handling

- Simple Alert
- Confirmation Alert
- Prompt Alert

Window Handling

- Multiple Windows
- Multiple Tabs

Actions Class

- Mouse Hover
- Double Click
- Drag and Drop
- Slider Handling

Web Tables

- Row Validation
- Data Verification

File Upload

- Single File Upload
- Multiple File Upload

Other Modules

- Dropdown Handling
- Date Picker
- Broken Links
- Search Functionality

Automated Test Cases:

Form Validation

1. Verify Name Field
2. Verify Email Field
3. Verify Phone Field
4. Verify Address Field

Radio Buttons

5. Verify Male Radio Button

Checkboxes

6. Verify All Day Checkboxes

Dropdowns

7. Verify Country Dropdown
8. Verify Color Dropdown
9. Verify Animal Dropdown

Date Picker

10. Verify Date Selection

File Upload

11. Verify Single File Upload
12. Verify Multiple File Upload

Search Functionality

13. Verify Search

Alert Handling

14. Verify Simple Alert
15. Verify Confirm Alert
16. Verify Prompt Alert

Window Handling

17. Verify New Tab
18. Verify Popup Window

Actions Class

19. Verify Mouse Hover
20. Verify Double Click
21. Verify Drag and Drop
22. Verify Slider

Web Tables

23. Verify Table Rows

Links

24. Verify Broken Links

General

25. Verify Page Title

Automation Test Report:

After successful execution of the automation suite, TestNG generates a detailed execution report that provides information about the overall test execution status. The report shown above demonstrates that all automated test cases executed successfully without any failures or skipped tests.

File

C:/Users/karun/git/CapstoneProject/Capstone_Project/test-output/Capstone%20Suite/Automation%20Practice%20Tests.html

Automation Practice Tests

Tests passed/Failed/Skipped: 25/0/0

Started on: Sun Jun 14 00:25:58 IST 2026

Total time: 241 seconds (241720 ms)

Included groups:

Excluded groups:

(Hover the method name to see the test class name)

PASSED TESTS				
Test method	Attribute(s)	Exception	Time (seconds)	Instance
verifyBrokenLinks Test class: testcases.AutomationPracticeTest		39	testcases.AutomationPracticeTest@131ef10	
verifyPopupWindowHandling Test class: testcases.AutomationPracticeTest		0	testcases.AutomationPracticeTest@131ef10	
verifyMouseHover Test class: testcases.AutomationPracticeTest		0	testcases.AutomationPracticeTest@131ef10	
verifyCountryDropdown Test class: testcases.AutomationPracticeTest		0	testcases.AutomationPracticeTest@131ef10	
verifySimpleAlert Test class: testcases.AutomationPracticeTest		0	testcases.AutomationPracticeTest@131ef10	
verifyCheckboxes Test class: testcases.AutomationPracticeTest		5	testcases.AutomationPracticeTest@131ef10	
verifySearchFunctionality Test class: testcases.AutomationPracticeTest		0	testcases.AutomationPracticeTest@131ef10	
verifyWindowHandling Test class: testcases.AutomationPracticeTest		0	testcases.AutomationPracticeTest@131ef10	
verifyEmailField Test class: testcases.AutomationPracticeTest		0	testcases.AutomationPracticeTest@131ef10	

Defects Identified:

The following validation defects were observed:

- Name field accepts numeric values.
- Email field accepts invalid email format.
- Mobile number accepts less than 10 digits.
- Address accepts special characters only.
- Broken Link Returns 404 Error

These defects indicate missing input validation in the application. This aligns with the defect analysis documented in the uploaded report.

Defect Report:

BUG-01: Name Field Accepts Numeric Values Without Validation

Steps to Reproduce:

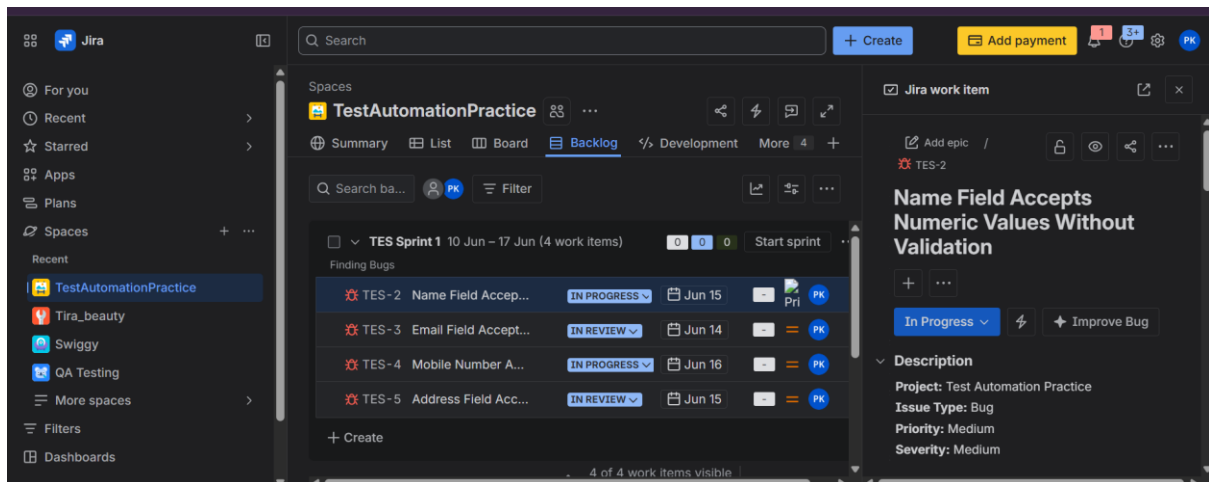
1. Open the application.
2. Enter 123456 in the Name field.
3. Submit the form.

Expected Result:

The Name field should accept only alphabetic characters.

Actual Result:

Numeric values are accepted without any validation message



BUG-02: Email Field Accepts Invalid Email Format

Steps to Reproduce:

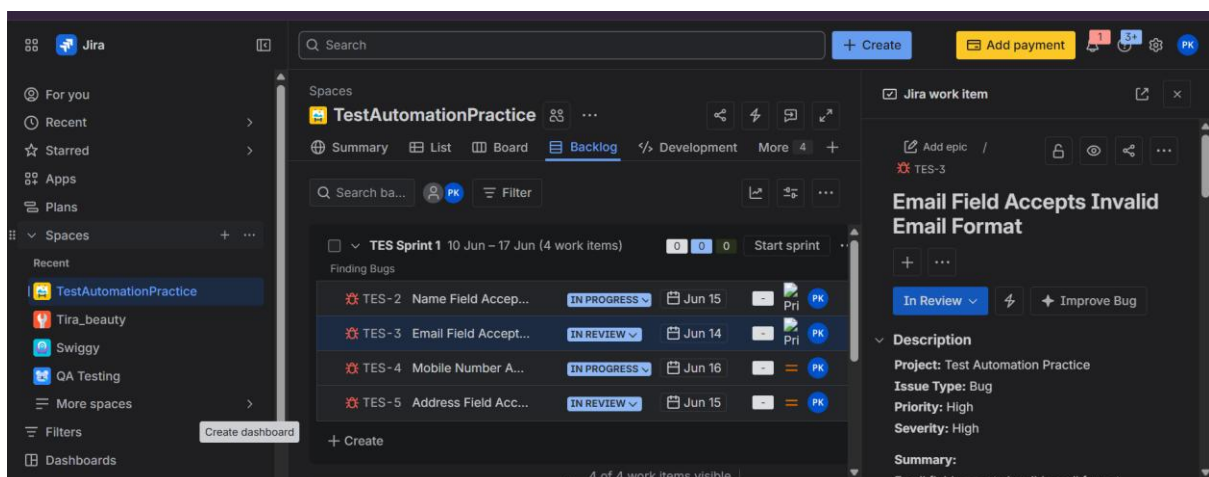
1. Open the application.
2. Enter karunakar123 in the Email field.
3. Submit the form.

Expected Result:

The Email field should accept only valid email formats containing @ and domain.

Actual Result:

Invalid email is accepted without validation.



BUG-03: Mobile Number Accepts Less Than 10 Digits

Steps to Reproduce:

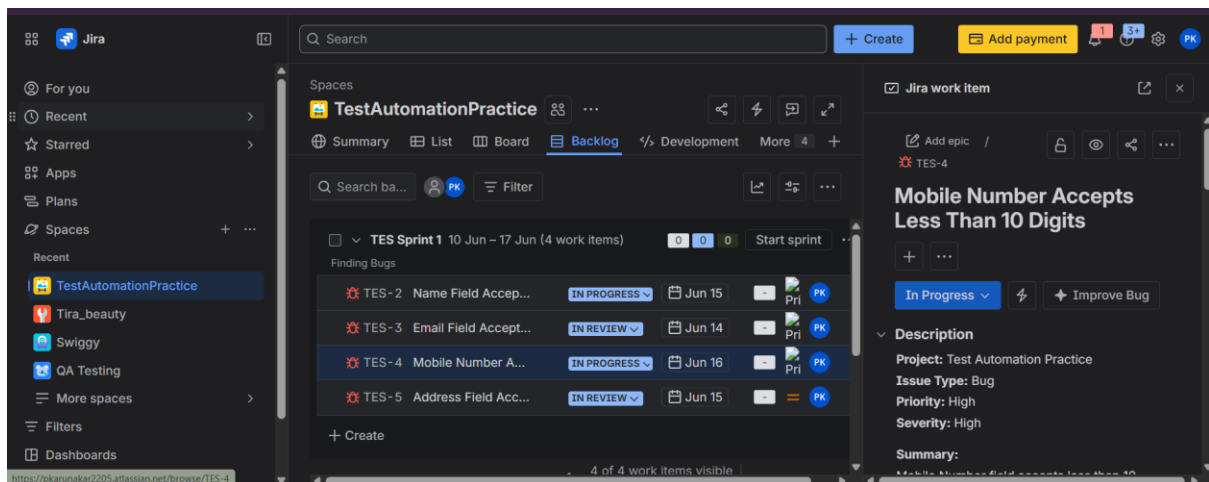
1. Open the application.
2. Enter 12345 in the Mobile Number field.
3. Submit the form.

Expected Result:

The Mobile Number field should accept exactly 10 digits.

Actual Result:

Application accepts less than 10 digits without validation.



BUG-04: Address Field Accepts Special Characters Only

Steps to Reproduce:

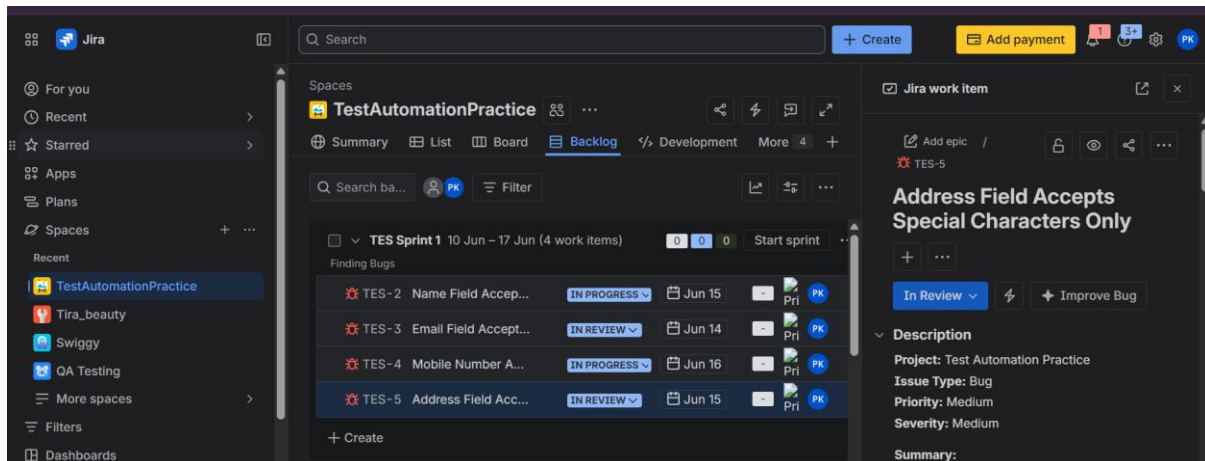
1. Open the application.
2. Enter @#\$%^&*() in the Address field.
3. Submit the form.

Expected Result:

Address field should contain meaningful address information.

Actual Result:

Special characters are accepted without validation.



BUG-05: Broken Link Returns 404 Error

Steps to Reproduce:

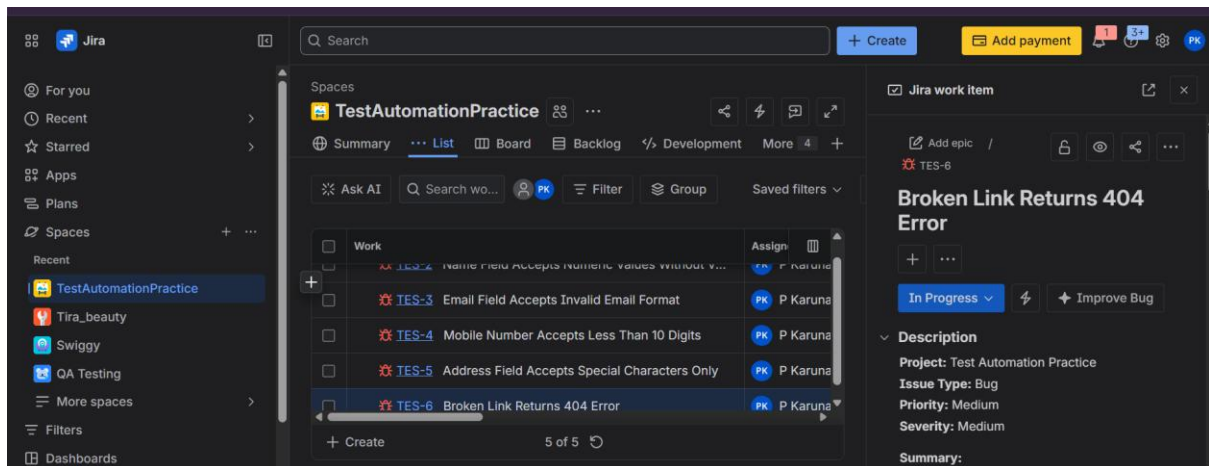
1. Navigate to the application.
2. Click available links or run broken link verification.
3. Observe response codes.

Expected Result:

All links should return valid HTTP response codes (200 OK).

Actual Result:

Some links return HTTP 404 Not Found.



Defect Summary Table:

Bug ID	Summary	Severity	Priority
BUG-01	Name field accepts numeric values	Medium	Medium
BUG-02	Invalid email accepted	High	High
BUG-03	Mobile number less than 10 digits accepted	High	High
BUG-04	Address field accepts special characters	Medium	Medium
BUG-05	Broken link returns 404	Medium	Medium

Jenkins Integration:

Jenkins was integrated with GitHub to achieve Continuous Integration.

Execution Flow:

GitHub Repository



Jenkins Pipeline



Maven Build



TestNG Execution



Report Generation



Build Status

Benefits:

- Automated execution
- Continuous testing
- Faster feedback
- Easy monitoring
- Build history maintenance

Jenkins / CapstoneProject / #2 / Stages

✓ < #2

Rerun

Started by P Karunakar

Started 6 min 29 sec ago

Queued 0.11 sec

Took 4 min 52 sec

Artifacts

Start

Checkout SCM

Checkout

Build

Post Actions

End

Search

Checkout SCM 3s

Checkout 2s

Build 4m 38s

Post Actions 0.48s

Post Actions 0.48s

Started 2m ago

Jenkins

Archive the artifacts 0.29s

Build Successful 75ms

Build Successful

Jenkins 2.555.2

Jenkins / CapstoneProject / #1

Status

Changes

Console Output

Edit Build Information

Delete build '#1'

Timings

Git Build Data

Pipeline Overview

Restart from Stage

Replay

Pipeline Steps

Workspaces

Next Build

✓ #1 (13-Jun-2026, 12:34:59 am)

Add description

Keep this build forever

Started 8 hr 27 min ago

Took 4 min 14 sec

Build Artifacts

Started by user P Karunakar

This run spent:

- 12 ms waiting;
- 4 min 14 sec build duration;
- 4 min 14 sec total from scheduled to completion.

Revision: aeb738d9a0d9e8bd5f105c9a8f90db28d2dd460a

Repository: https://github.com/PKarunakar2205/CapstoneProject.git

- refs/remotes/origin/main

No changes.

REST API Jenkins 2.555.2

GitHub Repository:

Repository URL:

<https://github.com/PKarunakar2205/CapstoneProject>

GitHub is used for:

- Source Code Management
- Version Control
- Collaboration
- Jenkins Integration

Test Execution Summary:

Metric	Count
Total Test Cases	25
Executed Test Cases	25
Passed Test Cases	25
Failed Test Cases	0
Blocked Test Cases	0

Challenges Faced:

- Jenkins configuration and setup
- Maven build issues
- File path handling
- Apache POI Excel integration
- Selenium locator maintenance
- Browser compatibility issues
- GitHub integration
- Dynamic element synchronization

Conclusion:

The Automation Testing Framework successfully automated the major functionalities of the Test Automation Practice Website using Selenium WebDriver, Java, TestNG, Maven, Jenkins, GitHub, and Apache POI.

The framework follows industry-standard Page Object Model architecture and provides reusable, maintainable, and scalable automation components. Continuous Integration was achieved using Jenkins, and version control was managed through GitHub.

The framework improved execution efficiency, reduced manual effort, increased test coverage, and enabled reliable regression testing. All planned automation scenarios were successfully implemented and executed.